

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Dama Yohitesh Naveen Sai (1BM23CS085)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Machine Learning (23CS6PCMAL)” carried out by **Dama Yohitesh Naveen Sai (1BM23CS085)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Lab faculty Incharge Name Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	21-2-2025	Write a python program to import and export data using Pandas library functions	4-8
2	3-3-2025	Demonstrate various data pre-processing techniques for a given dataset	9-14
3	10-3-2025	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	15-22
4	17-3-2025	Build Logistic Regression Model for a given dataset	23-24
5	24-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	25-27
6	7-4-2025	Build KNN Classification model for a given dataset.	28-30
7	21-4-2025	Build Support vector machine model for a given dataset	31-33
8	5-5-2025	Implement Random forest ensemble method on a given dataset.	34-36
9	5-5-2025	Implement Boosting ensemble method on a given dataset.	37-40
10	12-5-2025	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	41-44
11	12-5-2025	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	45-48

Github Link: <https://github.com/Yohitesh/ML-LAB>

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot

4/2/2026, LAB-1

1.) Load the DataSet. Ans) No. of columns = 5

Code:

```
from sklearn.datasets import load_iris
import pandas as pd
data = load_iris()
df = pd.DataFrame(data.data, column = data.feature_names)
df["species"] = data.target
df.head()
```

Output

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

2.) Understand the shape

Code:

```
df.shape
```

Ans) No. of rows = 150
No. of columns = 5

Output:

```
(150, 5)
```

3.) Explore columns, which column is the output?

Code:

```
df.columns
df.info()
```

Ans) The column "species" is the output column.

4) Which feature has the largest value?

→ `df.describe()`

Ans) The feature 'sepal length (cm)' has the largest value.

5) Are classes balanced?

→ `df['species'].value_counts()`

Ans) Yes, the classes are balanced, with each class having 50 samples.

6) Plot the graph. Do you see group forming?

→ `plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])`

`plt.xlabel('sepal length')`

`plt.ylabel('petal length')`

`plt.show()`

Ans) Yes, we can clearly see the group forming.

7) Compare two species. Do features values differ?

→ `df[df['species'] == 0].head()`

`df[df['species'] == 1].head()`

Ans) Yes, for species 0, the values are slightly less or lower whereas for species 1, the input values are on the higher side.

Ex for species 0, the petal length is around 1.5 cm whereas it is 4.5 cm for species 1.

Only the sepal width is almost similar for all different species.

8) Which features are related?

→ `df.corr()`

Ans) → sepal length & petal length : Correlation = 0.87
petal width & petal length : Correlation = 0.96
petal width & sepal length : Correlation = 0.81

Since, here the correlation is almost 1,
we can say that these features are positively correlated.

9) Sort the data. What happens to 'species' label?

→ `df.sort_values(by='petal length (cm)').head()`

Ans) When we sort the data by petal length, we can see the data samples with species = 0 at the top,
Thus, we can infer that the data samples with species = 0 have the smallest petal length.

~~o/p~~

(P, 0.0205) ← 0.572

Code:

```
In [ ]: import pandas as pd
        from sklearn.datasets import load_iris
        data = load_iris()
        df = pd.DataFrame(data.data, columns=data.feature_names)
        df["species"] = data.target
        df.head()
```

```
Out[ ]:   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
0              5.1           3.5           1.4           0.2           0
1              4.9           3.0           1.4           0.2           0
2              4.7           3.2           1.3           0.2           0
3              4.6           3.1           1.5           0.2           0
4              5.0           3.6           1.4           0.2           0
```

```
In [ ]: df.shape
```

```
Out[ ]: (150, 5)
```

```
In [ ]: df.columns
        df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   species                150 non-null   int64   
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

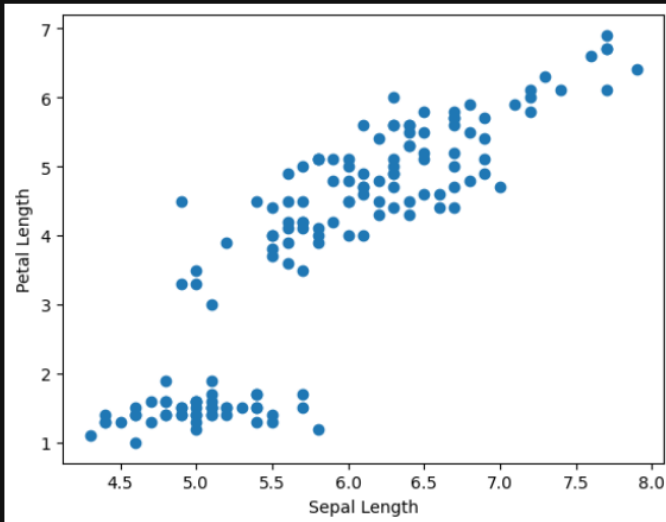
```
In [ ]: df.describe()
```

```
Out[ ]:   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
count      150.000000      150.000000      150.000000      150.000000      150.000000
mean         5.843333         3.057333         3.758000         1.199333         1.000000
std          0.828066         0.435866         1.765298         0.762238         0.819232
min          4.300000         2.000000         1.000000         0.100000         0.000000
25%          5.100000         2.800000         1.600000         0.300000         0.000000
50%          5.800000         3.000000         4.350000         1.300000         1.000000
75%          6.400000         3.300000         5.100000         1.800000         2.000000
max          7.900000         4.400000         6.900000         2.500000         2.000000
```

```
In [ ]: df["species"].value_counts()
```

```
Out[ ]:   count
species
0         50
1         50
2         50
```

```
In [ ]: import matplotlib.pyplot as plt
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel('Sepal Length')
plt.ylabel('Petal Length')
plt.show()
```



```
In [ ]: df[df['species'] == 0].head()
df[df['species'] == 1].head()
```

```
Out [ ]:   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
50                7.0                3.2                4.7                1.4         1
51                6.4                3.2                4.5                1.5         1
52                6.9                3.1                4.9                1.5         1
53                5.5                2.3                4.0                1.3         1
54                6.5                2.8                4.6                1.5         1
```

```
In [ ]: df.corr()
```

```
Out [ ]:   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
sepal length (cm)    1.000000    -0.117570    0.871754    0.817941    0.782561
sepal width (cm)    -0.117570    1.000000    -0.428440    -0.366126    -0.426658
petal length (cm)    0.871754    -0.428440    1.000000    0.962865    0.949035
petal width (cm)    0.817941    -0.366126    0.962865    1.000000    0.956547
species             0.782561    -0.426658    0.949035    0.956547    1.000000
```

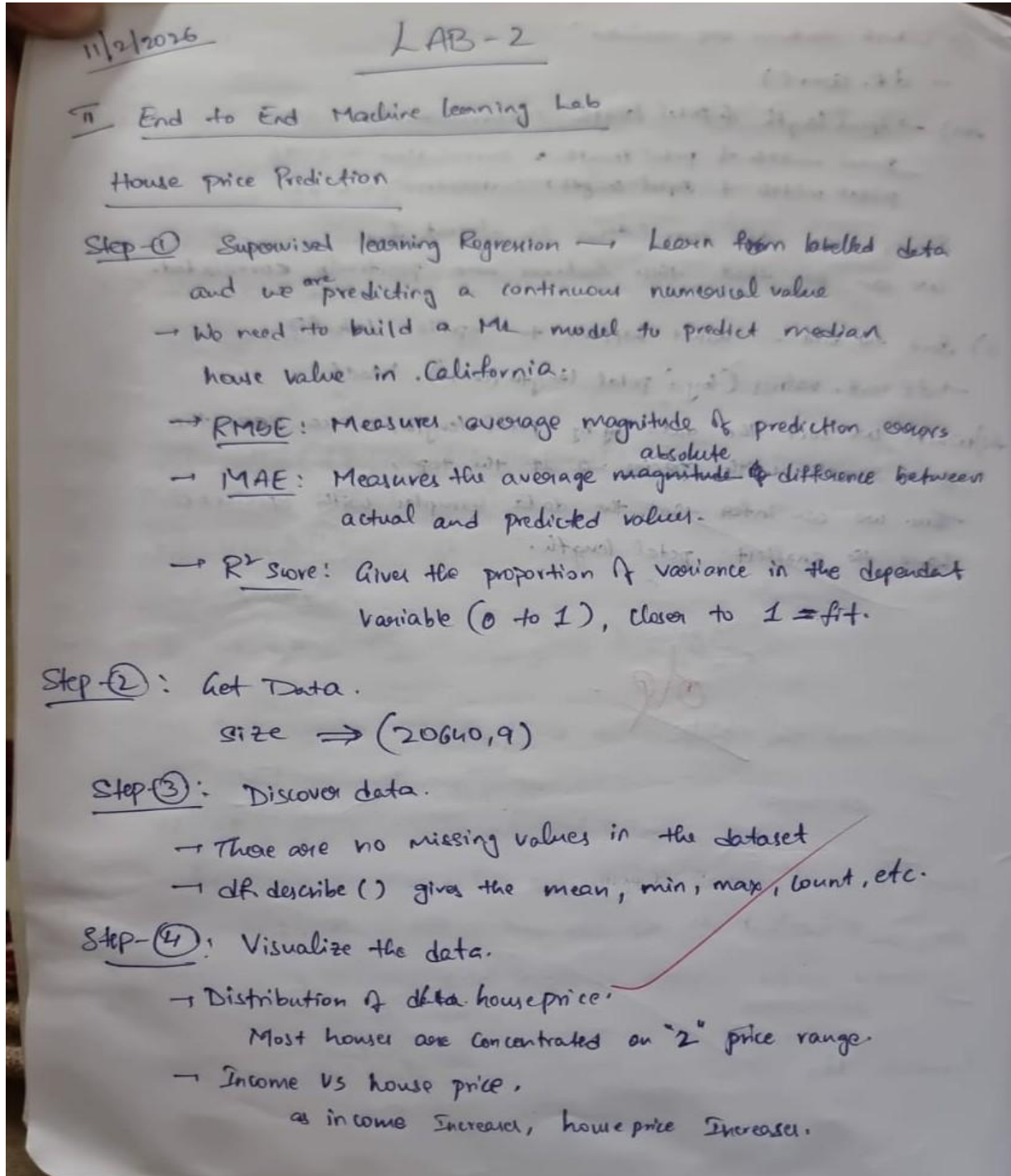
```
In [ ]: df.sort_values(by='petal length (cm)').head()
```

```
Out [ ]:   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
22                4.6                3.6                1.0                0.2         0
13                4.3                3.0                1.1                0.1         0
14                5.8                4.0                1.2                0.2         0
35                5.0                3.2                1.2                0.2         0
16                5.4                3.9                1.3                0.4         0
```


Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot:



→ Correlation heatmap

Med Inc has strongest positive correlation with med houseval

→ Location Visualization

Step 5: Preparing the data.

Train test split

80% of data for training

20% of data for testing

Feature scaling → required for linear regression

converts data to mean = 0

std = 1

- Because some features have large values (population) and some small.

Step 6: Select & train Base model

Linear Regression

RMSE = 0.7449

MAE = 0.5330

$R^2 = 0.5765$

Step 7: Train advanced models

* Decision Tree Regressor

RMSE = 0.7138

Since RMSE is lower, better performance than linear regression.

* Random Forest regression

RMSE = 0.5129

better performance

Step 8: Fine-tune the model

Hyperparameter tuning

RMSE = 0.5132

Step-9: Model comparison

Model	RMSE
Linear regression	0.744954
Decision Tree	0.713838
Random forest	0.512864
Tuned Random forest	0.513263

- Random forest $\rightarrow$ Lowest RMSE

Step-10 (1) Which model performed best

$\hookrightarrow$ Tuned Random forest performed best because it had the lowest RMSE

Hyper parameter tuning improved it.

(2) Why did random forest outperform linear regression

$\hookrightarrow$ Because linear regression assumes linear relationships.

$\hookrightarrow$ Knowing prices depend on complex non-linear relationships.

$\hookrightarrow$ Random forest captures non-linear patterns

(3) How did tuning improve performance

$\hookrightarrow$ Hyper parameter tuning

- found optimal tree depth

- reduced overfitting

- improved generalization

(4) What additional data could improve predictions

$\hookrightarrow$ Distance to city centres

$\hookrightarrow$ Crime rates

$\hookrightarrow$ Nearby facilities

Code:

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
In [ ]: # Load California housing dataset
housing = fetch_california_housing(as_frame=True)
# Create DataFrame
df = housing.frame
# Display basic info
df.head()
df.shape
df.info()
```

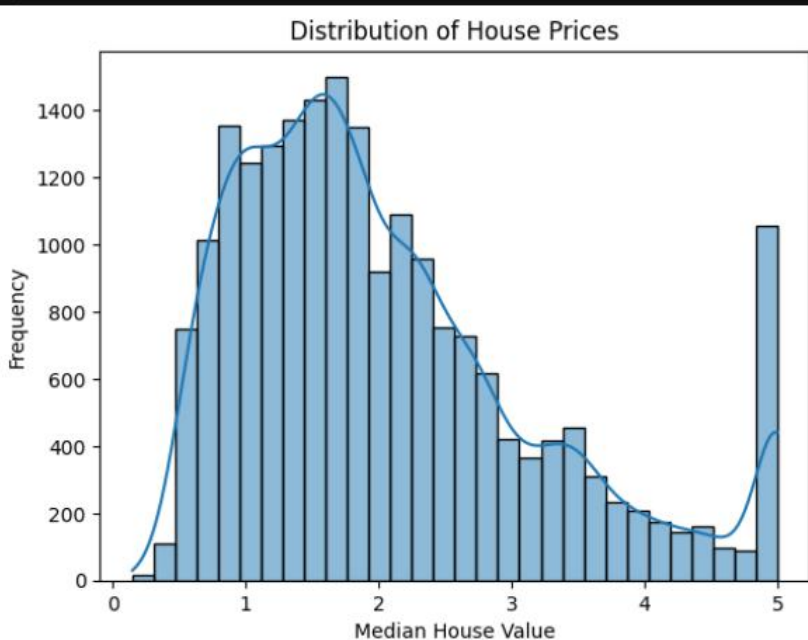
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   MedInc          20640 non-null  float64
1   HouseAge        20640 non-null  float64
2   AveRooms        20640 non-null  float64
3   AveBedrms       20640 non-null  float64
4   Population      20640 non-null  float64
5   AveOccup        20640 non-null  float64
6   Latitude        20640 non-null  float64
7   Longitude       20640 non-null  float64
8   MedHouseVal     20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

```
In [ ]: df.describe()
# Check missing values
df.isnull().sum()
```

```
Out[ ]:      0
MedInc    0
HouseAge  0
AveRooms  0
AveBedrms 0
Population 0
AveOccup  0
Latitude  0
Longitude  0
MedHouseVal 0
```



```
In [ ]: # Distribution of House prices
plt.figure()
sns.histplot(df['MedHouseVal'], bins=30, kde=True)
plt.xlabel('Median House Value')
plt.ylabel('Frequency')
plt.title('Distribution of House Prices')
plt.show()
```



```
In [ ]: # Income VS House Price
plt.figure()
sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'], alpha=0.5)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Income vs House Price')
plt.show()
```



```
In [ ]: # Step 7.1: Decision Tree Regressor
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

rmse_dt
```

Out[]: np.float64(0.7138377373947808)

```
In [ ]: # Step 7.2: Random Forest Regressor
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

rmse_rf
```

Out[]: np.float64(0.5128644706054181)

```
In [23]: # Step 8.1: Hyperparameter Tuning
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    rf, param_grid, cv=3,
    scoring='neg_mean_squared_error'
)

grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_

y_pred_best = best_model.predict(X_test)
rmse_best = np.sqrt(mean_squared_error(y_test, y_pred_best))

rmse_best
```

Out[23]: np.float64(0.5132629837109113)

```
In [24]: # Step 9: Model Comparison
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Decision Tree', 'Random Forest', 'Tuned Random Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})

results
```

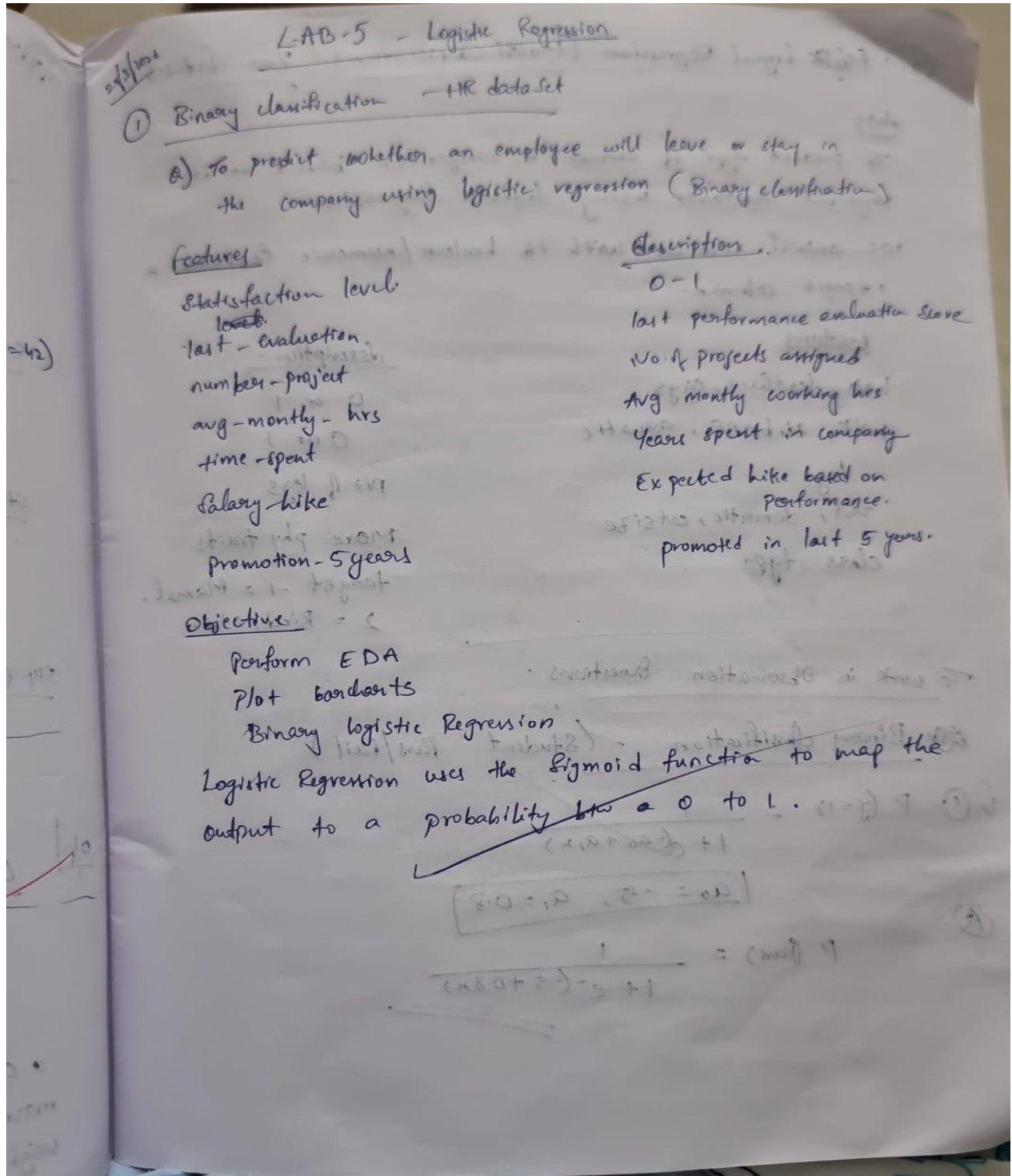
Out[24]:

	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot:



② ~~Multiple~~ Logistic Regression (Multi classification) (Zoo dataset).

Aim

To predict type of animal (7 categories) based on its physical features using logistic Regression

101 animal records with 16 boolean/numeric features, a 1 target column.

Features

hair, feathers, eggs

milk, airborne, aquatic

legs

tail, domestic, catsize

class type

descriptions

0 or 1

0 or 1

no. of legs

More phy traits

target - 1 = Mammal,

2 = Bird, etc.

To write in Observation Questions.

~~Q1~~ Binary classification - (Student Pass/Fail)

$$Q1) P(y=1) = \frac{1}{1 + e^{-(a_0 + a_1 x)}}$$

$$[a_0 = -5, a_1 = 0.8]$$

$$P(\text{Pass}) = \frac{1}{1 + e^{-(-5 + 0.8x)}}$$

Ref.)

Q2) Prob for 7 hours study

$$z = a_0 + a_1 x = 0.6$$

$$p(\text{pass}) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + 0.5488} = 0.6457$$

64.57%

Q3) Soft max function for $z = [2, 1, 0]$

$$p(z_i) = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$e^2 = 7.3$$

$$e^1 = 2.7$$

$$e^0 = 1$$

$$\text{Sum} = 7.38 + 2.7 + 1.0 = 1$$

Multi classification

class	1	7.3 / 11.1	66.57%
	2	2.7 / 11.1	24.47%
	3	1.1 / 11.1	9.9%

Total 1.0

Sum
1.0

Code:

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, roc_curve, roc_auc_score
)

import warnings
warnings.filterwarnings("ignore")
print("✅ Libraries imported successfully!")
```

✅ Libraries imported successfully!

```
In [ ]: from google.colab import files

print("📁 Please select 'HR_comma_sep.csv' from your PC...")
uploaded = files.upload()  # + file picker popup appears here

# Automatically pick the uploaded filename
import io
filename = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[filename]))

print(f"✅ File '{filename}' loaded successfully!")
print(f"Shape : {df.shape}")
df.head()
```

📁 Please select 'HR_comma_sep.csv' from your PC...

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving HR_comma_sep.csv to HR_comma_sep.csv

✅ File 'HR_comma_sep.csv' loaded successfully!
Shape : (14999, 10)

```
Out [ ]: satisfaction_level  last_evaluation  number_project  average_monthly_hours  time_spend_company  Work_accident  left  promotion_last_5years  Department  salary
0          0.38             0.53             2              157                3                0    1                0      sales      low
1          0.80             0.86             5              262                6                0    1                0      sales  medium
2          0.11             0.88             7              272                4                0    1                0      sales  medium
3          0.72             0.87             5              223                5                0    1                0      sales      low
4          0.37             0.52             2              159                3                0    1                0      sales      low
```

```
In [ ]: print("=" * 55)
print(" DATASET INFO")
print("=" * 55)
df.info()

print("\n" + "=" * 55)
print(" MISSING VALUES")
print("=" * 55)
print(df.isnull().sum())

print("\n" + "=" * 55)
print(" STATISTICAL SUMMARY")
print("=" * 55)
print(df.describe())

print("\n" + "=" * 55)
print(" TARGET DISTRIBUTION (left : 0=Stayed, 1=Left)")
print("=" * 55)
print(df['left'].value_counts())
print(f"\n Retention Rate : {(df['left']==0).mean()*100:.1f}%")
print(f" Attrition Rate : {(df['left']==1).mean()*100:.1f}%")
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   satisfaction_level      14999 non-null  float64
1   last_evaluation         14999 non-null  float64
2   number_project          14999 non-null  int64
3   average_monthly_hours  14999 non-null  int64
4   time_spend_company      14999 non-null  int64
5   Work_accident           14999 non-null  int64
6   left                   14999 non-null  int64
7   promotion_last_5years   14999 non-null  int64
8   Department              14999 non-null  object
9   salary                 14999 non-null  object
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB

=====
MISSING VALUES
=====
satisfaction_level      0
last_evaluation         0
number_project          0
average_monthly_hours   0
time_spend_company      0
Work_accident           0
left                    0
promotion_last_5years   0
Department              0
salary                  0
dtype: int64

=====
STATISTICAL SUMMARY
=====
      satisfaction_level  last_evaluation  number_project \
count      14999.000000      14999.000000      14999.000000
mean         0.612834         0.716102         3.803054
std          0.248631         0.171169         1.232592
min          0.090000         0.360000         2.000000
25%          0.440000         0.560000         3.000000
50%          0.640000         0.720000         4.000000
75%          0.820000         0.870000         5.000000
max          1.000000         1.000000         7.000000

      average_monthly_hours  time_spend_company  Work_accident  left \
count      14999.000000      14999.000000      14999.000000      14999.000000
mean        201.050337         3.498233         0.144610         0.238083
std         49.943099         1.460136         0.351719         0.425924
min          96.000000         2.000000         0.000000         0.000000
25%         156.000000         3.000000         0.000000         0.000000
50%         200.000000         3.000000         0.000000         0.000000
75%         245.000000         4.000000         0.000000         0.000000
max         310.000000        10.000000         1.000000         1.000000

      promotion_last_5years
count      14999.000000
mean         0.021268
std          0.144281
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
max          1.000000

=====
TARGET DISTRIBUTION (left : 0=Stayed, 1=Left)
=====
left
0    11428
1     3571
Name: count, dtype: int64

Retention Rate : 76.2%
Attrition Rate : 23.8%

```

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay
)

import warnings
warnings.filterwarnings("ignore")
print("🟢 Libraries imported successfully!")
```

🟢 Libraries imported successfully!

```
In [ ]: from google.colab import files
import io

print("📁 Step 1 - Please select 'zoo-data.csv' from your PC...")
uploaded1 = files.upload()
zoo_filename = list(uploaded1.keys())[0]
zoo = pd.read_csv(io.BytesIO(uploaded1[zoo_filename]))
print(f"🟢 '{zoo_filename}' loaded! Shape: {zoo.shape}")

print("\n📁 Step 2 - Please select 'zoo-class_type.csv' from your PC...")
uploaded2 = files.upload()
class_filename = list(uploaded2.keys())[0]
class_df = pd.read_csv(io.BytesIO(uploaded2[class_filename]))
print(f"🟢 '{class_filename}' loaded! Shape: {class_df.shape}")
```

📁 Step 1 - Please select 'zoo-data.csv' from your PC...

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving zoo-data.csv to zoo-data (1).csv

🟢 'zoo-data (1).csv' loaded! Shape: (181, 18)

📁 Step 2 - Please select 'zoo-class_type.csv' from your PC...

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving zoo-class-type.csv to zoo-class-type (1).csv

🟢 'zoo-class-type (1).csv' loaded! Shape: (7, 4)

```
In [ ]: print("=" * 55)
print(" ZOO DATA - First 5 Rows")
print("=" * 55)
print(zoo.head())

print("\n" + "=" * 55)
print(" CLASS TYPE FILE")
print("=" * 55)
print(class_df)
```

```
=====
ZOO DATA - First 5 Rows
=====
  animal_name  hair  feathers  eggs  milk  airborne  aquatic  predator  \
0  aardvark     1         0     0     1         0         0         1
1  antelope     1         0     0     1         0         0         0
2    bass      0         0     1     0         0         1         1
3    bear      1         0     0     1         0         0         1
4    boar       1         0     0     1         0         0         1

  toothed  backbone  breathes  venomous  fins  legs  tail  domestic  catsize  \
0         1         1         1         0     0     4         0         1
1         1         1         1         0     0     4         1         0
2         1         1         0         0     1     0         1         0
3         1         1         1         0     0     4         0         1
4         1         1         1         0     0     4         1         1

  class_type
0          1
1          1
2          4
3          1
4          1
=====
CLASS TYPE FILE
=====
  Class_Number  Number_Of_Animal_Species_In_Class  Class_Type  \
0             1                               41      Mammal
1             2                               20       Bird
2             3                               5      Reptile
3             4                               13       Fish
4             5                               4  Amphibian
5             6                               8        Bug
6             7                               10  Invertebrate

  Animal_Names
0  aardvark, antelope, bear, boar, buffalo, calf,...
1  chicken, crow, dove, duck, flamingo, gull, haw...
2  pitviper, seasnake, slowworm, tortoise, tuatara
3  bass, carp, catfish, chub, dogfish, haddock, h...
4  frog, frog, newt, toad
```

In []:

```
print("=" * 55)
print(" ZOO DATASET INFO")
print("=" * 55)
zoo.info()

print("\n" + "=" * 55)
print(" MISSING VALUES")
print("=" * 55)
print(zoo.isnull().sum())

print("\n" + "=" * 55)
print(" CLASS TYPE DISTRIBUTION")
print("=" * 55)
print(zoo['class_type'].value_counts())

print("\n" + "=" * 55)
print(" STATISTICAL SUMMARY")
print("=" * 55)
print(zoo.describe())
```

ZOO DATASET INFO

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101 entries, 0 to 100
Data columns (total 18 columns):
#   Column          Non-Null Count  Dtype
---  --
0   animal_name      101 non-null    object
1   hair             101 non-null    int64
2   feathers         101 non-null    int64
3   eggs             101 non-null    int64
4   milk             101 non-null    int64
5   airborne         101 non-null    int64
6   aquatic          101 non-null    int64
7   predator         101 non-null    int64
8   toothed          101 non-null    int64
9   backbone         101 non-null    int64
10  breathes         101 non-null    int64
11  venomous         101 non-null    int64
12  fins             101 non-null    int64
13  legs             101 non-null    int64
14  tail             101 non-null    int64
15  domestic         101 non-null    int64
16  catsize          101 non-null    int64
17  class_type       101 non-null    int64
dtypes: int64(17), object(1)
memory usage: 14.3+ KB
```

MISSING VALUES

```
animal_name    0
hair           0
feathers       0
eggs           0
milk           0
airborne       0
aquatic        0
predator       0
toothed        0
backbone       0
breathes       0
venomous       0
fins           0
legs           0
tail           0
domestic       0
catsize        0
class_type     0
dtype: int64
```

CLASS TYPE DISTRIBUTION

```
class_type
1    41
2    20
4    13
7    10
6     8
3     5
5     4
Name: count, dtype: int64
```

STATISTICAL SUMMARY

```
count    hair    feathers    eggs    milk    airborne    aquatic \
mean    0.425743    0.198020    0.584158    0.405041    0.237624    0.356436
std     0.496921    0.400495    0.495325    0.493522    0.427750    0.481135
min     0.000000    0.000000    0.000000    0.000000    0.000000    0.000000
25%     0.000000    0.000000    0.000000    0.000000    0.000000    0.000000
50%     0.000000    0.000000    1.000000    0.000000    0.000000    0.000000
75%     1.000000    0.000000    1.000000    1.000000    0.000000    1.000000
max     1.000000    1.000000    1.000000    1.000000    1.000000    1.000000

count    predator    toothed    backbone    breathes    venomous    fins \
mean    0.554455    0.603960    0.821782    0.792079    0.079208    0.168317
std     0.499505    0.491512    0.384605    0.407844    0.271410    0.376013
min     0.000000    0.000000    0.000000    0.000000    0.000000    0.000000
25%     0.000000    0.000000    1.000000    1.000000    0.000000    0.000000
50%     1.000000    1.000000    1.000000    1.000000    0.000000    0.000000
75%     1.000000    1.000000    1.000000    1.000000    0.000000    0.000000
max     1.000000    1.000000    1.000000    1.000000    1.000000    1.000000

count    legs    tail    domestic    catsize    class_type
mean    2.841584    0.742574    0.128713    0.435644    2.831683
std     2.033385    0.439397    0.336552    0.498314    2.102709
min     0.000000    0.000000    0.000000    0.000000    1.000000
25%     2.000000    0.000000    0.000000    0.000000    1.000000
50%     4.000000    1.000000    0.000000    0.000000    2.000000
75%     4.000000    1.000000    0.000000    1.000000    4.000000
max     8.000000    1.000000    1.000000    1.000000    7.000000
```

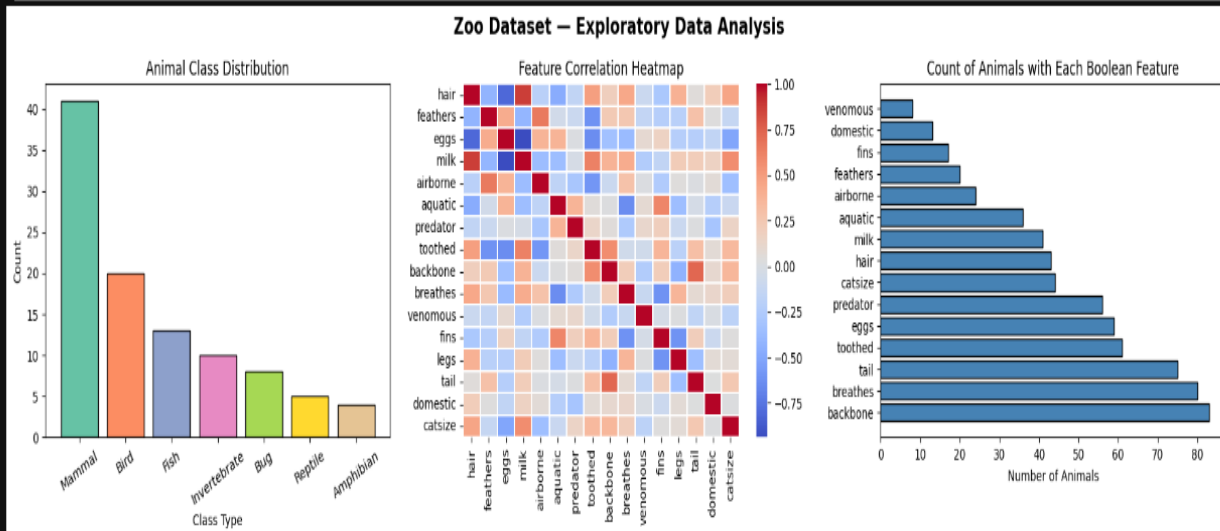
```
In [ ]:
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("Zoo Dataset - Exploratory Data Analysis",
            fontsize=15, fontweight='bold')

# 1. Class type distribution
class_counts = zoo['class_label'].value_counts()
axes[0].bar(class_counts.index, class_counts.values,
            color=sns.color_palette('Set2', len(class_counts)), edgecolor='black')
axes[0].set_title("Animal Class Distribution")
axes[0].set_xlabel("Class Type")
axes[0].set_ylabel("Count")
axes[0].tick_params(axis='x', rotation=30)

# 2. Feature correlation heatmap
num_zoo = zoo.drop(columns=['animal_name', 'class_type', 'class_label'],
                    errors='ignore').select_dtypes(include=np.number)
sns.heatmap(num_zoo.corr(), cmap='coolwarm', ax=axes[1],
            linewidth=0.3, annot=False)
axes[1].set_title("Feature Correlation Heatmap")

# 3. Boolean features - how many animals have each feature
bool_cols = [c for c in num_zoo.columns if num_zoo[c].nunique() == 2]
feat_sums = num_zoo[bool_cols].sum().sort_values(ascending=False)
axes[2].bar(feat_sums.index, feat_sums.values,
            color='steelblue', edgecolor='black')
axes[2].set_title("Count of Animals with Each Boolean Feature")
axes[2].set_xlabel("Number of Animals")

plt.tight_layout()
plt.show()
```



```
In [ ]:
drop_cols = ['animal_name', 'class_label']
df_model = zoo.drop(columns=[c for c in drop_cols if c in zoo.columns])

# Features and Target
target = 'class_type'
features = [c for c in df_model.columns if c != target]

X = df_model[features]
y = df_model[target]

print("Features used:", features)
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"Classes: {sorted(y.unique())} + {[class_map.get(i, 1) for i in sorted(y.unique())]}")

# Check if any preprocessing needed (already binary features mostly)
print("\nData types:")
print(X.dtypes.value_counts())
print(f"No missing values: {X.isnull().sum().sum() == 0}")

# Features used: ['hair', 'feathers', 'eggs', 'milk', 'airborne', 'aquatic', 'predator', 'toothed', 'backbone', 'breathes', 'venomous', 'fins', 'legs', 'tail', 'domestic', 'catsize']

X shape: (181, 16)
y shape: (181,)

Classes: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7)] + ['Mammal', 'Bird', 'Reptile', 'Fish', 'Amphibian', 'Bug', 'Invertebrate']

Data types:
int64 16
Name: count, dtype: int64
```


Program 4

Build Logistic Regression Model for a given dataset

Screenshot:

```
⇒ CART Dataset - 2 Regression

from sklearn.datasets import load_diabetes
data = load_diabetes
X = data.data
Y = data.target

Train test split
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42)

Decision Tree Regression (CART)
reg = DecisionTreeRegressor()
reg.fit(X_train, Y_train)
Y_pred = reg.predict(X_test)

print(mean_squared_error(Y_test, Y_pred))
print(mean_absolute_error(Y_test, Y_pred))
print(r2_score(Y_test, Y_pred))

O/P
MSE: 4593.359
MAE: 52.0112
R2 Score: 0.1330
```

Code:

```
In [13]: #Decision Tree Regression (CART)
#California Housing Dataset

housing = fetch_california_housing()

X = housing.data
y = housing.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

regressor = DecisionTreeRegressor(criterion="squared_error")
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
```

```
In [9]: mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nCalifornia Housing Results")

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
```

```
California Housing Results
MSE: 0.57026127025625
RMSE: 0.7551564541578454
MAE: 0.48054355135658916
R2 Score: 0.5767612834091143
```

```
In [11]: # Diabetes Dataset

from sklearn.datasets import load_diabetes

diabetes = load_diabetes()

X = diabetes.data
y = diabetes.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

regressor2 = DecisionTreeRegressor()
regressor2.fit(X_train, y_train)

y_pred2 = regressor2.predict(X_test)

mse = mean_squared_error(y_test, y_pred2)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred2)
r2 = r2_score(y_test, y_pred2)

print("\nDiabetes Dataset Results")

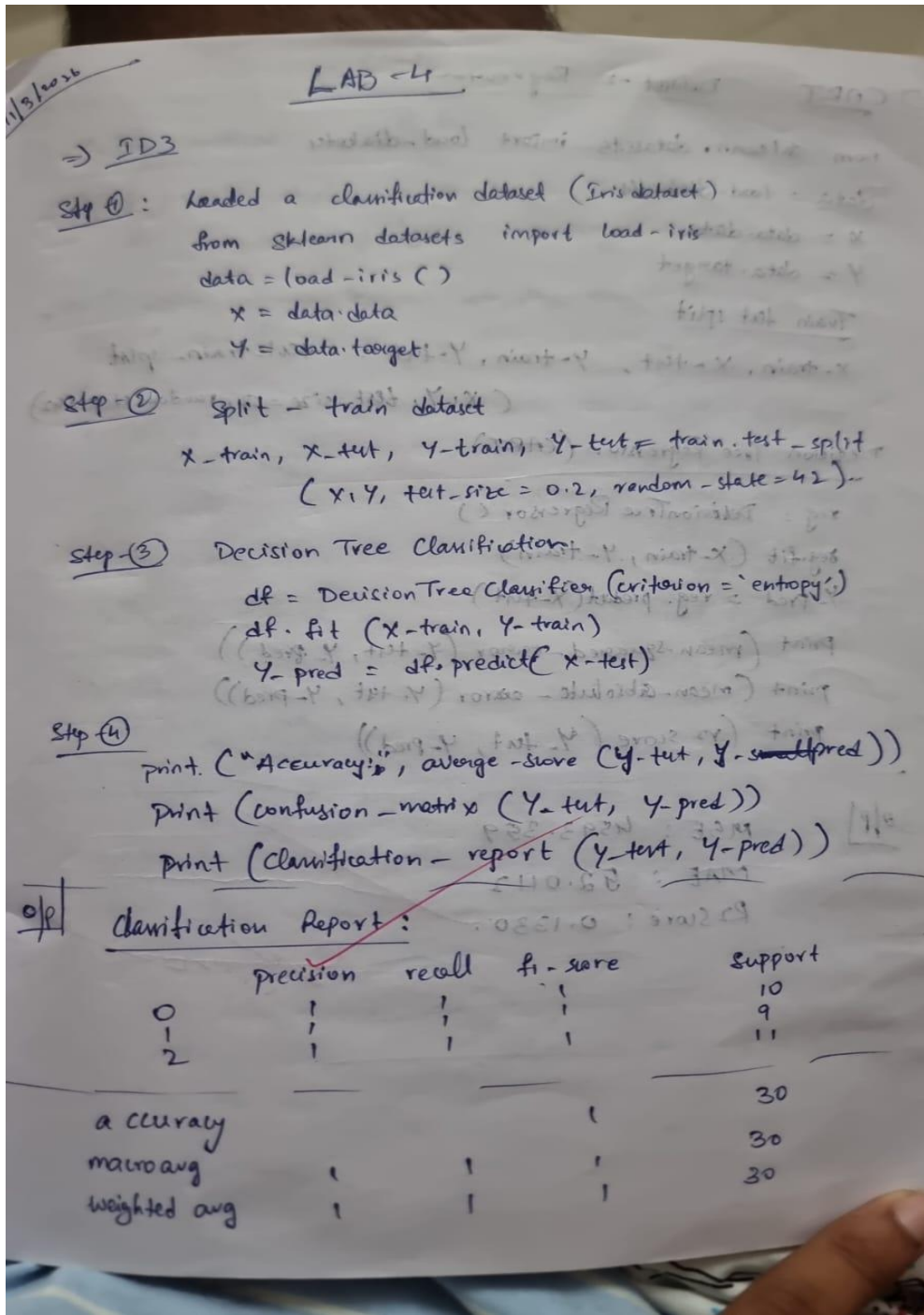
print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
```

```
Diabetes Dataset Results
MSE: 7700.5842696629215
RMSE: 87.75297299614938
MAE: 70.96629213483146
R2 Score: -0.09818964601200464
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot:



Code:

```
In [3]: import pandas as pd
import numpy as np
from sklearn.datasets import load_iris, fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
```

```
In [15]: # Decision Tree Classification (ID3)
# Iris Dataset

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier(criterion="entropy")
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
In [5]: print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='macro'))
print("Recall:", recall_score(y_test, y_pred, average='macro'))
print("F1 Score:", f1_score(y_test, y_pred, average='macro'))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
In [14]: # Wine Dataset

from sklearn.datasets import load_wine

wine = load_wine()

X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model2 = DecisionTreeClassifier(criterion="entropy")
model2.fit(X_train, y_train)

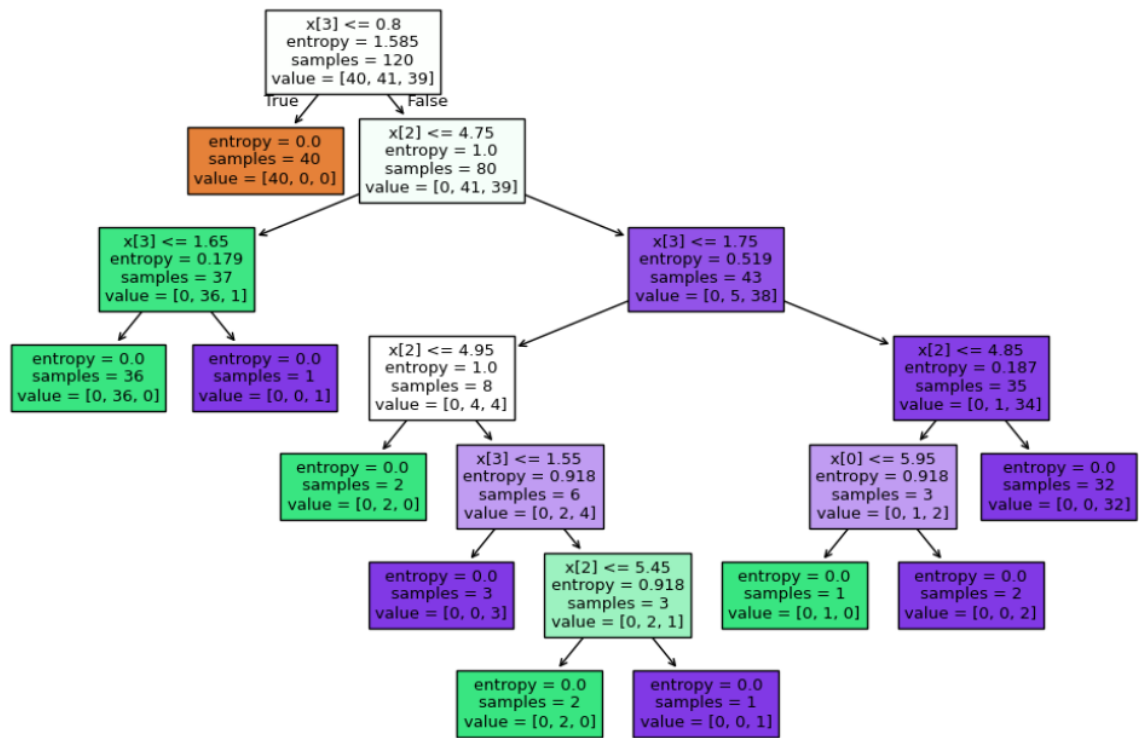
y_pred2 = model2.predict(X_test)

print("\nWine Dataset Results")

print("Accuracy:", accuracy_score(y_test, y_pred2))
print("Precision:", precision_score(y_test, y_pred2, average='macro'))
print("Recall:", recall_score(y_test, y_pred2, average='macro'))
print("F1 Score:", f1_score(y_test, y_pred2, average='macro'))
```

```
Wine Dataset Results
Accuracy: 0.9722222222222222
Precision: 0.9814814814814815
Recall: 0.9583333333333334
F1 Score: 0.9682539682539684
```

```
In [7]: plt.figure(figsize=(12,8))
plot_tree(model, filled=True)
plt.show()
```



Program 6

Build KNN Classification model for a given dataset.

Screenshot:

25/2/26 LAB-3

⇒ KNN Regression Algorithm

Step-① We should choose a value for K first in order to determine the nearest neighbours to make the prediction.

Step-② After choosing K , we have to calculate distance for each data point in the training data with the target data point.

Step-③ We can use Euclidean distance formula to find the distance between the points.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + \dots}$$

Step-④ After finding the distances for all the data points, sort all the data records in the training data set in ascending order of the distances.

Step-⑤

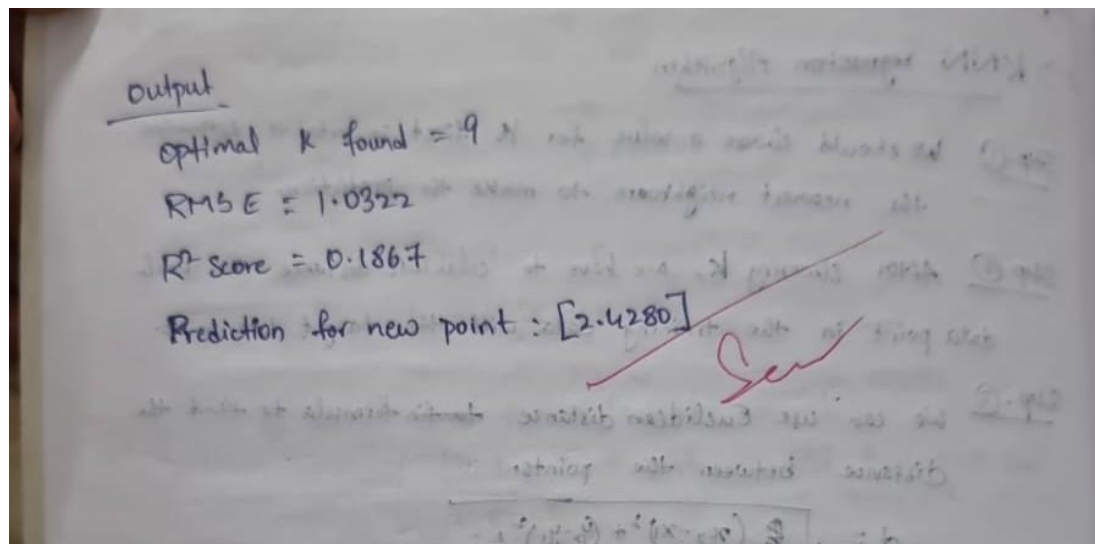
Choose the topmost K number of records, for either regression or classification.

↳ If its for classification,

Assign the new data entry to the major class in the topmost K^{th} rows (nearest neighbors).

↳ If its for Regression,

After choosing the nearest neighbors, calculate the mean or average for their distances and assign its distance to the point.



Code:

```
In [2]: #KNN Regression program

import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Step 0: Load dataset
housing = fetch_california_housing()
X = housing.data
y = housing.target

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 1: Elbow Method to find optimal k
errors = []
k_values = range(1, 31)

for k in k_values:
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred_val = knn.predict(X_test)
    mse = mean_squared_error(y_test, y_pred_val)
    errors.append(mse)
```

```

error.append(mse)

# Plot error vs k
plt.plot(k_values, errors, marker='o')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Mean Squared Error')
plt.title('Elbow Method for KNN Regression')
plt.show()

# Step 2: Choose optimal k (minimum error)
optimal_k = k_values[np.argmin(errors)]
print("Optimal k found:", optimal_k)

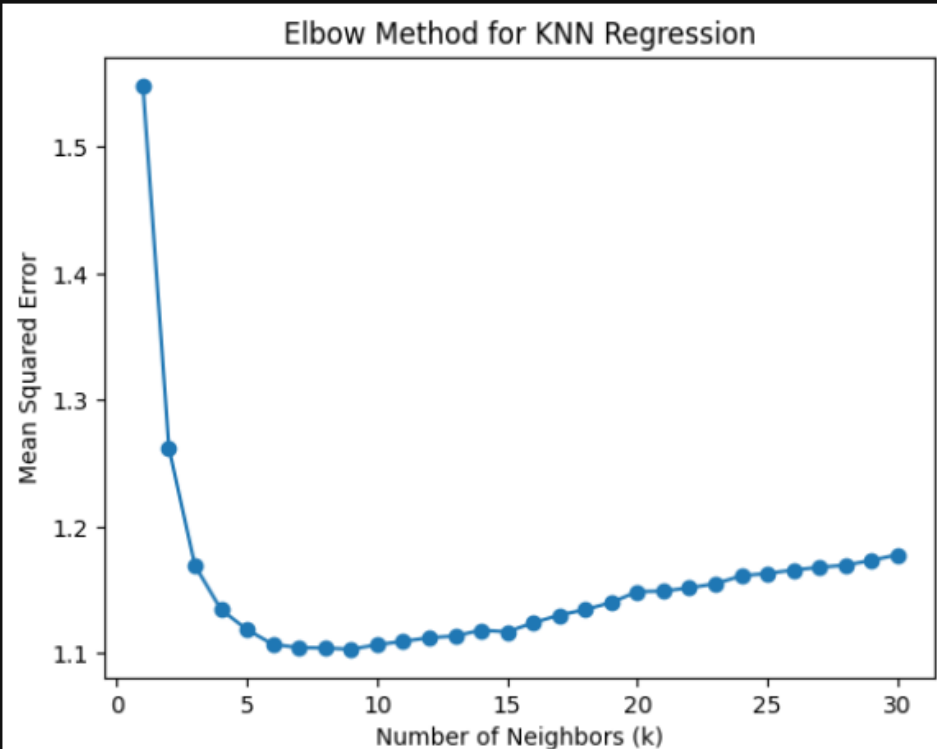
# Step 3: Train final KNN model with optimal k
knn = KNeighborsRegressor(n_neighbors=optimal_k, weights='distance')
knn.fit(X_train, y_train)

# Step 4: Predict on test set
y_pred = knn.predict(X_test)

# Step 5: Evaluate performance
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R² Score:", r2_score(y_test, y_pred))

# Step 6: Predict for a NEW point (example)
# Suppose we want to predict for a new house with given features:
new_point = np.array([[8.3252, 41.0, 6.9841, 1.0238, 322.0, 2.5556, 37.88, -122.23]])
prediction = knn.predict(new_point)
print("Prediction for new point:", prediction)

```



```

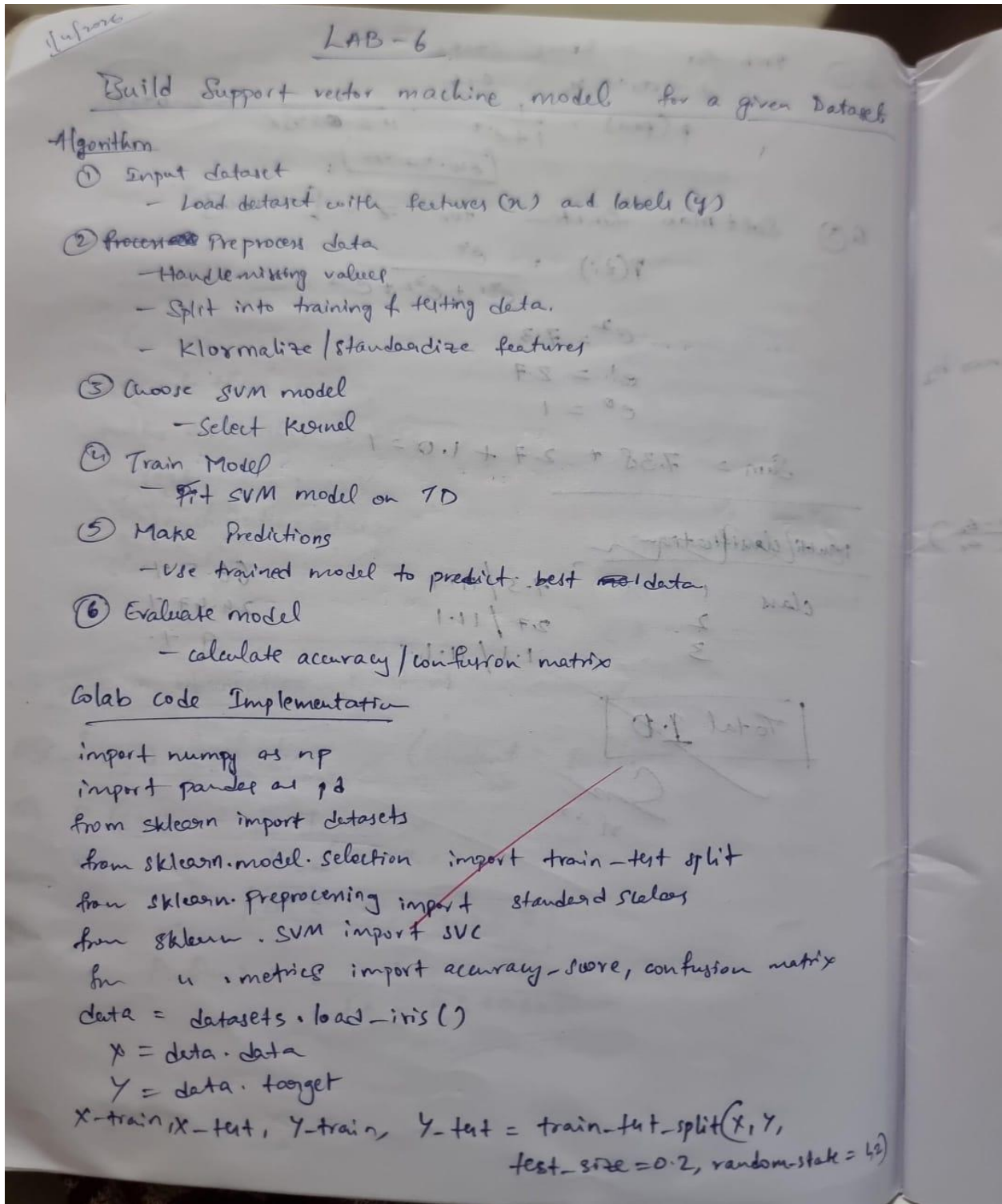
Optimal k found: 9
RMSE: 1.0322938633531986
R² Score: 0.1867953913539413
Prediction for new point: [2.42805487]

```

Program 7

Build Support vector machine model for a given dataset

Screenshot:



```
scale = StandardScaler()
```

$X_{\text{train}} = \text{scaler.fit_transform}(X_{\text{train}})$

$$x_{\text{test}} = \text{scaler.transform}(x_{\text{test}})$$

model = SVC (kernel = "linear")

```
model = svm (
    model.fit (X_train, Y_train)
```

$$y_{\text{pred}} = \text{model.predict}(x_{\text{test}})$$

Accuracy = accuracy-score (y - tat , y - pred)

can = confusion-matrix (y-test, y-pred)

```
print(Accuracy, ' ', accuracy)
```

```
print("Confusion matrix", cm)
```

e/p:

Predicted values: $\begin{bmatrix} 1 & 0 & 2 & 1 & 0 & 1 & 2 & 0 & 0 & 0 \\ 1 & 2 & 1 & 2 & 2 & 2 & 2 \end{bmatrix}$

Actual values: [1021101211200001211221222222]

Accuracy: 1.D

~~Confusion matrix: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$~~

Code:

```
In [ ]: # Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Load dataset
data = datasets.load_iris()
X = data.data
y = data.target

# Feature scaling (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset (fixed for same results every time)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:\n", cr)
```

Accuracy: 0.9666666666666667

Confusion Matrix:

```
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```

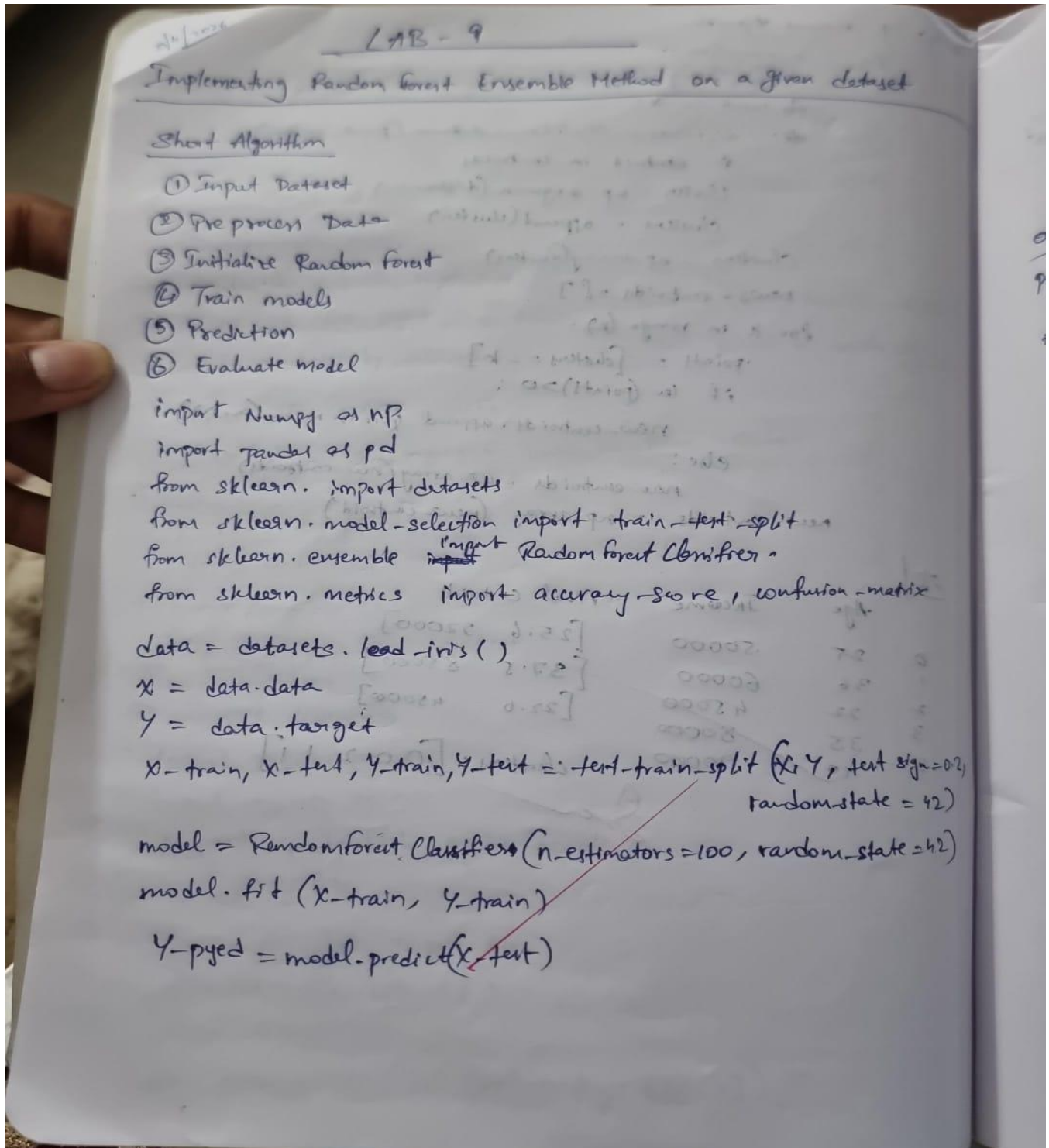
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot:




```

print("predicted:", y-pred)
print("Actual:", y-test)
print("Accuracy:", accuracy-score(y-test, y-pred))
print("Confusion matrix", confusion-matrix(y-test, y-pred))

```

o/p)

predicted = [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 2 1 2 2 2 2 2 2]

Actual = [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 2 1 2 2 2 2 2 2]

Accuracy = 1.0

Confusion matrix = $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

✓

Code:

```
In [ ]: import pandas as pd
        from sklearn.datasets import load_iris
        from sklearn.model_selection import train_test_split
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.metrics import accuracy_score, confusion_matrix

        iris = load_iris()

        X = iris.data
        y = iris.target

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )

        model = RandomForestClassifier(n_estimators=100, random_state=42)

        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)

        accuracy = accuracy_score(y_test, y_pred)
        print("Accuracy:", accuracy)

        cm = confusion_matrix(y_test, y_pred)
        print("Confusion Matrix:\n", cm)
```

Accuracy: 1.0

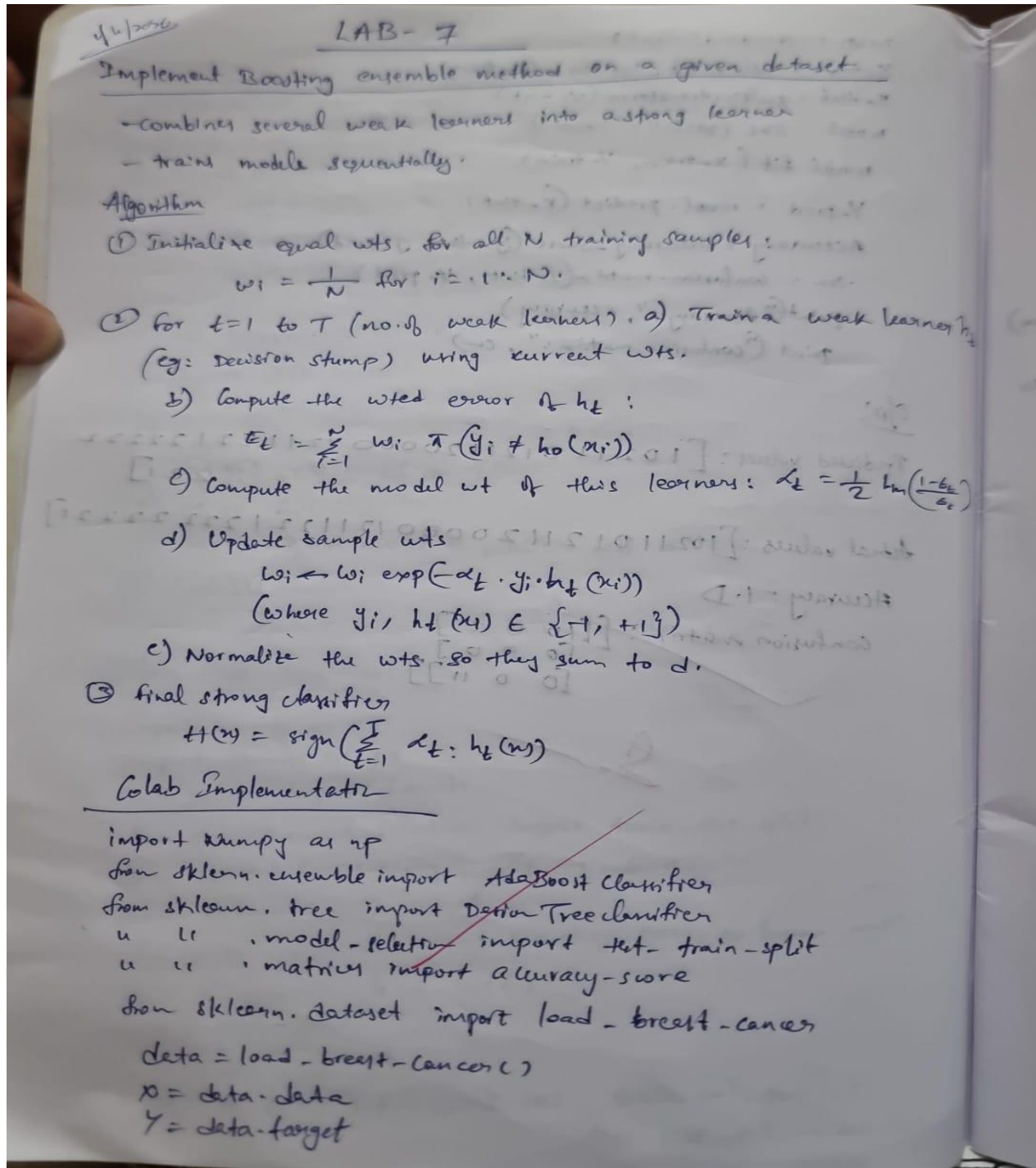
Confusion Matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot:



$y = np.where(y == 0, -1, 1)$
 $x_train, x_test, y_train, y_test = train_test_split(x, y,$
 $\text{test_size} = 0.2, \text{random_state} = 42)$

$\text{base_estimator} = \text{Decision Tree Classifier}(\text{max_depth} = 1)$

$\text{model} = \text{AdaBoost Classifier}(\text{estimator} = \text{base_estimator},$
 $n_estimators = 50, \text{learning_rate} = 1.0, \text{random_state} = 42)$

$\text{model.fit}(x_train, y_train)$

$y_pred = \text{model.predict}(x_test)$

$\text{Accuracy} = \text{accuracy_score}(y_test, y_pred)$

o/p

Test Accuracy: 0.9737

No. of weak learners used = 50

Sum

Code:

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
In [ ]: # Load dataset
data = load_breast_cancer()

# Convert to DataFrame
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# Preview
print(X.head())
print("Target values:", np.unique(y))
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0      17.99      10.38      122.80      1001.0      0.11840
1      20.57      17.77      132.90      1326.0      0.08474
2      19.69      21.25      130.00      1203.0      0.10960
3      11.42      20.38       77.58       386.1      0.14250
4      20.29      14.34      135.10      1297.0      0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0      0.27760      0.3001      0.14710      0.2419
1      0.07864      0.0869      0.07017      0.1812
2      0.15990      0.1974      0.12790      0.2069
3      0.28390      0.2414      0.10520      0.2597
4      0.13280      0.1980      0.10430      0.1809

   mean fractal dimension  ...  worst radius  worst texture  worst perimeter  \
0      0.07871  ...      25.38      17.33      184.60
1      0.05667  ...      24.99      23.41      158.80
2      0.05999  ...      23.57      25.53      152.50
3      0.09744  ...      14.91      26.50       98.87
4      0.05883  ...      22.54      16.67      152.20

   worst area  worst smoothness  worst compactness  worst concavity  \
0      2019.0      0.1622      0.6656      0.7119
1      1956.0      0.1238      0.1866      0.2416
2      1709.0      0.1444      0.4245      0.4504
3       567.7      0.2098      0.8663      0.6869
4      1575.0      0.1374      0.2050      0.4000

   worst concave points  worst symmetry  worst fractal dimension
0      0.2654      0.4601      0.11890
1      0.1860      0.2750      0.08902
2      0.2430      0.3613      0.08758
3      0.2575      0.6638      0.17300
4      0.1625      0.2364      0.07678

[5 rows x 30 columns]
Target values: [0 1]
```

target values: [0 1]

```
In [ ]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
```

```
In [ ]: # Weak Learner (Decision Stump)
        base_model = DecisionTreeClassifier(max_depth=1)

        # AdaBoost model
        model = AdaBoostClassifier(
            estimator=base_model,
            n_estimators=50,
            learning_rate=1.0,
            random_state=42
        )

        # Train
        model.fit(X_train, y_train)
```

```
Out[ ]: AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1),
                           random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [ ]: y_pred = model.predict(X_test)

        print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy: 0.9649122807017544

```
In [ ]: print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

        print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Confusion Matrix:

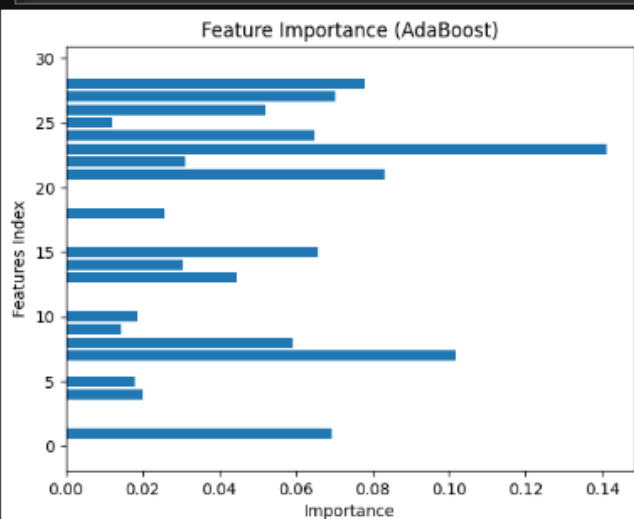
```
[[40  3]
 [ 1 70]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

```
In [ ]: importance = model.feature_importances_

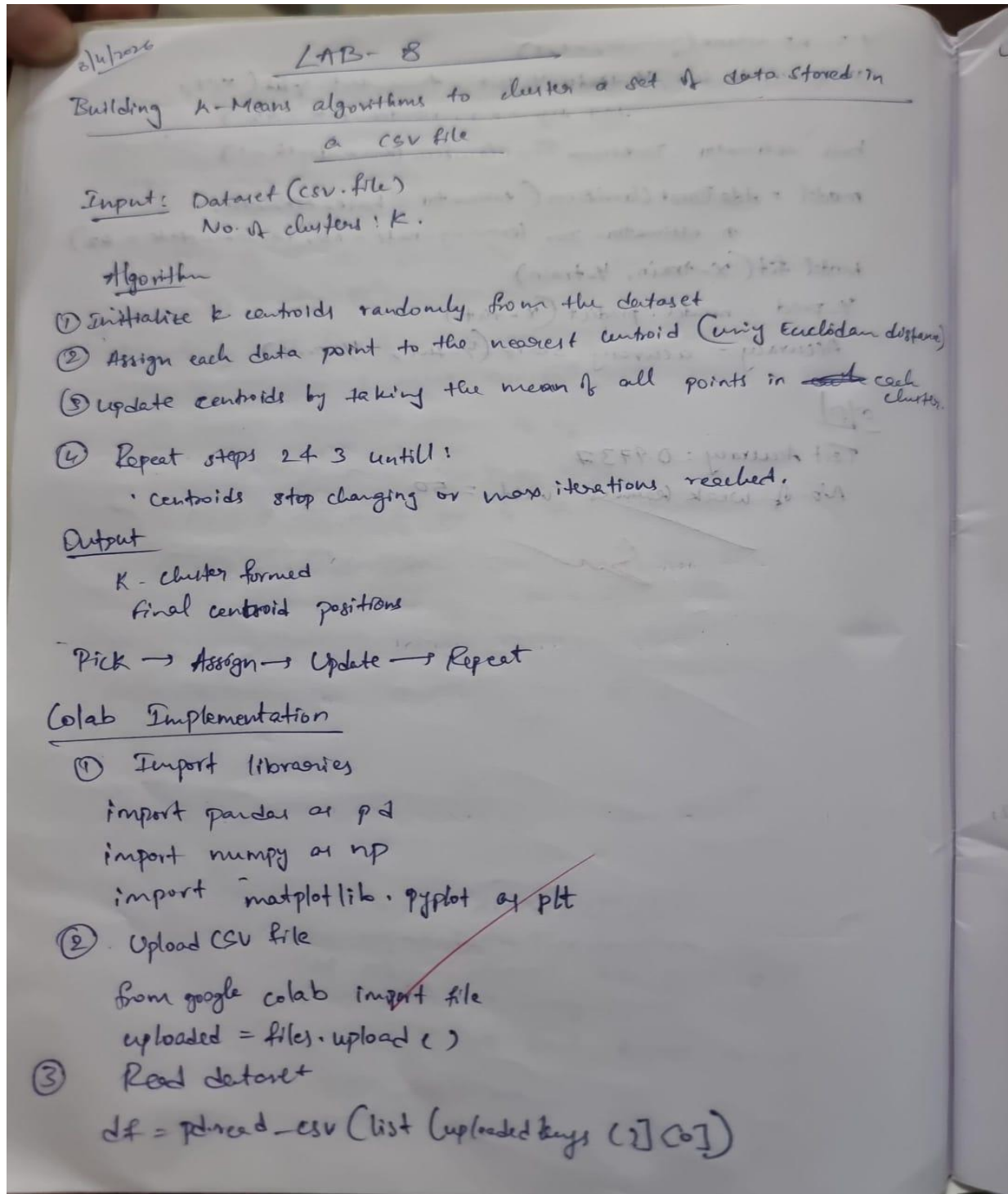
        plt.figure()
        plt.barh(range(len(importance)), importance)
        plt.xlabel("Importance")
        plt.ylabel("Features Index")
        plt.title("Feature Importance (AdaBoost)")
        plt.show()
```



Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot:



4) K-means Algorithm
 for i in range(100):
 clusters = []
 for point in X:
 distances = euclidean_distance(point, centers)
 for centroid in centroids:
 cluster = np.argmin(distances)
 clusters = append(cluster)
 clusters = np.array(clusters)
 new_centers = []
 for k in range(K):
 points = [clusters == k]
 if len(points) > 0:
 new_centers.append(points.mean(axis))
 else:
 new_centers = np.array(new_centers)
 new_centers = np.array(new_centers)

O/P:

	Age	Income	Final centroid
0	25	50000	[25.6 52000]
1	36	60000	[37.5 85000]
2	22	45000	[22.0 45000]
3	35	80000	
4	40	90000	

 clusters: [0 0 2 1 1]

See

Code:

```
In [ ]: import pandas as pd
import numpy as np

# Create synthetic dataset
np.random.seed(42)

data = {
    "Age": np.random.randint(18, 60, 100),
    "Income": np.random.randint(20000, 100000, 100),
    "SpendingScore": np.random.randint(1, 100, 100)
}

df = pd.DataFrame(data)

# Save as CSV
df.to_csv("customer_data.csv", index=False)

print("Dataset created successfully!")
df.head()
```

Dataset created successfully!

```
Out[ ]:   Age  Income  SpendingScore
```

0	56	28392	19
1	46	50535	20
2	32	98603	96
3	25	72256	71
4	38	55222	52

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("customer_data.csv")

# Select features (Income & Spending Score)
X = df[['Income', 'SpendingScore']].values

# Number of clusters
k = 4

# Initialize centroids randomly
np.random.seed(42)
centroids = X[np.random.choice(X.shape[0], k, replace=False)]

# Function to calculate Euclidean distance
def distance(a, b):
    return np.linalg.norm(a - b)
```

```

# K-Means algorithm
for iteration in range(100):
    clusters = [[] for _ in range(k)]

    # Assign points to nearest centroid
    for point in X:
        distances = [distance(point, centroid) for centroid in centroids]
        cluster_index = np.argmin(distances)
        clusters[cluster_index].append(point)

    # Store old centroids
    old_centroids = centroids.copy()

    # Update centroids
    for i in range(k):
        if clusters[i]:
            centroids[i] = np.mean(clusters[i], axis=0)

    # Stop if centroids don't change
    if np.all(old_centroids == centroids):
        break

# Convert clusters to numpy arrays
clusters = [np.array(cluster) for cluster in clusters]

```

```

In [ ]: colors = ['red', 'blue', 'green', 'purple']

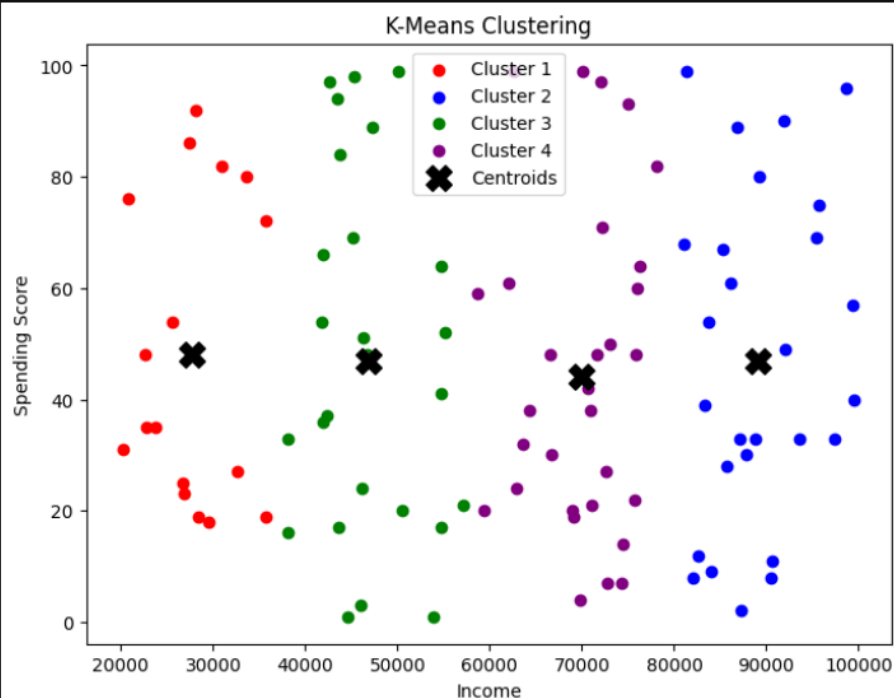
plt.figure(figsize=(8,6))

for i in range(k):
    if len(clusters[i]) > 0:
        plt.scatter(clusters[i][:,0], clusters[i][:,1],
                    c=colors[i], label=f'Cluster {i+1}')

# Plot centroids
plt.scatter(centroids[:,0], centroids[:,1],
            c='black', marker='X', s=200, label='Centroids')

plt.xlabel("Income")
plt.ylabel("Spending Score")
plt.title("K-Means Clustering")
plt.legend()
plt.show()

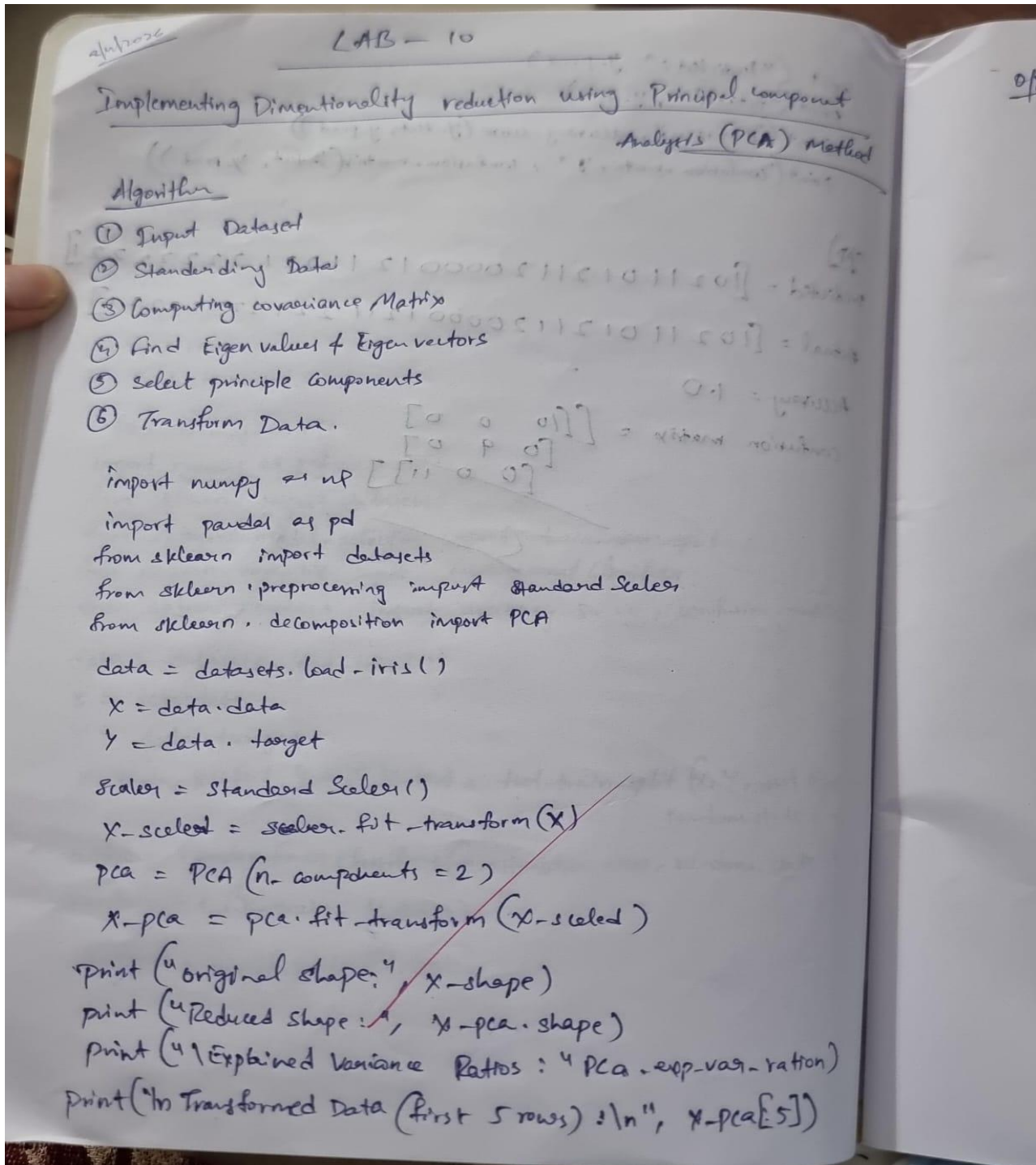
```



Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot:



op/

original shape : (150, 4)

Reduced shape : (150, 2)

Explained variance ratios : [0.72, 0.23]

Transformed Data (first 5 rows) :

([-2.26 0.48]

[-2.08 -0.67]

[-2.36 -0.34]

[-2.29 -0.59]

[-2.38 0.64])

Sept 18

Code:

```
In [ ]: # Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Step 2: Load dataset (Glass Identification Dataset)
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/glass/glass.data"

columns = ['Id', 'RI', 'Na', 'Mg', 'Al', 'Si', 'K', 'Ca', 'Ba', 'Fe', 'Type']
data = pd.read_csv(url, names=columns)

print("Dataset Shape:", data.shape)
print(data.head())

# Step 3: Drop ID column
data = data.drop('Id', axis=1)

# Step 4: Separate features and target
X = data.drop('Type', axis=1)
y = data['Type']

# Step 5: Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 6: Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Reduced Shape:", X_pca.shape)

# Step 7: Explained variance
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

```
# Step 8: Visualization
plt.figure()
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA - Glass Identification Dataset")
plt.colorbar(label='Glass Type')
plt.show()
```

Dataset Shape: (214, 11)

	Id	RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type
0	1	1.52101	13.64	4.49	1.10	71.78	0.06	8.75	0.0	0.0	1
1	2	1.51761	13.89	3.60	1.36	72.73	0.48	7.83	0.0	0.0	1
2	3	1.51618	13.53	3.55	1.54	72.99	0.39	7.78	0.0	0.0	1
3	4	1.51766	13.21	3.69	1.29	72.61	0.57	8.22	0.0	0.0	1
4	5	1.51742	13.27	3.62	1.24	73.08	0.55	8.07	0.0	0.0	1

Reduced Shape: (214, 2)

Explained Variance Ratio: [0.27901819 0.2277858]

